

Mybatis相关资料

一、mybatis的接口实现类是怎么被Spring管理

利用了Spring的扩展点，Bean工厂后处理器，默认情况下，Spring对扫描的通过@Component注解、@Service等注解注册为Bean的类会先将其放入到BeanDefinitionMap中，并且排除接口。因此首先重写Spring的扫描类ClassPathBeanDefinitionScanner对剔除接口的方法（isCandidateComponent）进行重写，通过实现BeanDefinitionRegistryPostProcessor，然后扫描获取接口，将这些接口注册为BeanDefinition，并放入到Map中，然后将BeanDefinition的BeanClass 通过FactoryBean替换成JDK动态代理的实例，将其改为其实现类。

关于BeanFactory和FactoryBean区别

BeanFactory：就是Spring的IOC容器

FacotryBean:是一个bean，但是它是一个特殊的bean，所以也是由BeanFactory来管理的。其是一个接口，他必须被一个**bean**去实现。不过FactoryBean不是一个普通的Bean，它会表现出工厂模式的样子,是一个能产生或者修饰对象生成的工厂Bean里面的getObject()就是用来获取FactoryBean产生的对象，而不是是用来的对象：如下产生的Bean对象是B的实例，而不再是A的实例

```
@Component
public void A implements BeanFactory {
    void getObject() {
        return B();
    }
}
```

所以在BeanFactory中使用“&”来得到FactoryBean本身（即getBean中使用&A获取到的就是本身，即a的实例）用来区分通过容器获取FactoryBean产生的对象还是获取FactoryBean本身。

并且getObject属于懒加载。

- 首先MyBatis的Mapper接口核心是JDK动态代理
- Spring会排除接口，无法注册到IOC容器中
- MyBatis 实现了BeanDefinitionRegistryPostProcessor 可以动态注册BeanDefinition 需要自定义扫描器(继承Spring内部扫描器ClassPathBeanDefinitionScanner)重写排除接口的方法
- 但是接口虽然注册成了BeanDefinition但是无法实例化Bean 因为接口无法实例化
- 需要将BeanDefinition的BeanClass 替换成JDK动态代理的实例(偷天换日)
- Mybatis 通过FactoryBean的工厂方法设计模式可以自由控制Bean的实例化过程，可以在getObject方法中创建JDK动态代理